Lecture 6: Tokenization

Week 2: From Characters to Meaningful Units

PSYC 51.07: Models of Language and Communication

Learning Objectives 🎯

```
By the end of this lecture, you will:

1. Understand what tokenization is and why it matters
```

2. Explain the limitations of word-level tokenization
3. Describe how subword tokenization works (BPE, WordPiece, SentencePiece)
4. Compare different tokenization methods and their trade-offs
5. Implement and experiment with different tokenizers using HuggingFace

Central question: How do we break text into meaningful units for AI models? 🔤

What is Tokenization? 🔍

```
**Definition:** Converting text into smaller units (tokens)



**Why tokenize?**
- Neural networks process numbers, not text
```

- Need discrete units to create vocabulary
- First step in any NLP pipeline

  What can tokens be?

  - **Characters:** a, b, c, ... (very fine-grained)
- **Words:** "hello", "world" (intuitive but limited)
- **Subwords:** "un", "happiness" (sweet spot! 🎯)

The Tokenization Spectrum 📊

```
1Fine-grained -> Coarse-grained -> Character -> Vocab: 100s -> Seq: Very long ->
```

```
**Key trade-off:** Vocabulary size vs. sequence length
```

Why Not Just Split on Spaces? 🤔

```
**The naive approach:**

    `text.split(" ")` → Done! 🎉



**Problem 1: Vocabulary explosion 💥**
- English: ~170,000 words in active use
```

- Each inflection counts separately: "run", "runs", "running", "ran"
- Compounds: "ice cream", "icecream", "ice-cream"

  Problem 2: Unknown words ❓

  - Out-of-vocabulary (OOV): "supercalifragilisticexpialidocious"

Word-level Limitations Illustrated 📉

```
1Word-level 📈 -> Subword-level 📊 -> Character-level → -> More data
```

```
**Observation:** Word-level vocabulary keeps growing!

Subword vocabulary stabilizes at reasonable size.
```

The OOV Problem in Action 🚨

```python
```

Simple word-level vocabulary

vocab = {"hello", "world", "the", "cat", "sat"}

def tokenize_word_level(text, vocab):
tokens = text.lower().split()
result = []
for token in tokens:
if token in vocab:
result.append(token)
else:

^^^ "jumped" is unknown!

```
21    - Fixed, manageable vocabulary (30k-50k tokens)
22- Handle unknown words by breaking into known parts
23- Capture morphology: prefixes, suffixes, stems
24- Language-agnostic: works for any language!
25
26
27
28    **How?** Learn common character sequences from data!
29
30---
31
32# Solving the OOV Problem: Concrete Example 🎯
33
34**The problem with word-level tokenization:**
35
```

...continued

Model can learn that 'super-' often means "very/above"

```
21            \node[rectangle, draw, fill=orange!30, minimum width=2c...
22-->
23
```

...continued

[Diagram placeholder - manual conversion required]

```
1
2
3
4
5
6    **Trade-off:** More tokens = longer sequences vs. smaller vocabulary
7
```

Model can learn that 'super-' often means "very/above"

```
21
22
23    **Algorithm:**
24    1. Start with character-level vocabulary
252. Count all adjacent pairs in corpus
263. Merge most frequent pair → create new token
274. Repeat until desired vocabulary size
28
29
30
31    **Used by:** GPT, GPT-2, GPT-3, RoBERTa, BART, many others!
32
33
34
35    📖 **Reference:** Sennrich, Haddow, & Birch (2016). Neural Machine Transla
36
```

Model can learn that 'super-' often means "very/above"

```
41
42    **Training data:** "low low low lower lower newest newest newest newest wi
43
44
45
46    **Initial tokens (characters):**
47
48        `l, o, w, e, r, n, s, t, i, d`
49
50
51
52
53    **Iteration 1:** Most frequent pair = `e, s` (appears 4 times)
54    - Merge → new token: `es`
55- Vocabulary: `l, o, w, e, r, n, s, t, i, d, es`
56
```

Model can learn that 'super-' often means "very/above"

```
61- Vocabulary: `l, o, w, e, r, n, s, t, i, d, es, est`
62
63
64
65    **Iteration 3:** Most frequent = `l, o`
66    - Merge → new token: `lo`
67
68
69
70    Continue until reaching target vocabulary size (e.g., 30,000)...
71
72---
73
74# BPE: Concrete Worked Example 📝
75
76**Let's trace through tokenizing "lowest" after training:**
```

Output: "I'm learning about tokenization!"

```
21
22
23    **Intuition:** Prefer merging pairs that are "surprisingly common" togethe
24
25
26
27    **Used by:** BERT, DistilBERT, Electra
28
29
30
31    **Special tokens:**
32    - `##` prefix for continuation subwords
33- Example: "playing" → `["play", "##ing"]`
34
35
36
37    📖 **Reference:** Wu et al. (2016). Google's Neural Machine Translation Sy
38
```

Output: "I'm learning about tokenization!"

```
41# WordPiece Example with BERT 🔍
42
43
```

...continued

from transformers import AutoTokenizer

BERT uses WordPiece

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I'm learning about tokenization!"

tokens = tokenizer.tokenize(text)

print("Tokens:", tokens)

Breaks into morphological components!

```
21
22
23
24    **Two algorithms:**
25    1. **Unigram Language Model:** Start with large vocabulary, iteratively re
262. **BPE:** Same as before, but on raw text
27
28
29
30    **Used by:** T5, ALBERT, XLNet, mT5 (multilingual models)
31
32
33
34    📖 **Reference:** Kudo & Richardson (2018). SentencePiece: A simple and la
35
36---
37
38# SentencePiece in Action 🚀
```

Breaks into characters/subwords without needing pre-tokenization

```
21
22
23
24
25    📖 **HuggingFace Resources:**
26    - [Chapter 2.4: Behind the Pipeline - Tokenizers](https://huggingface.co/l
27- [Chapter 6: Tokenizers (full chapter)](https://huggingface.co/learn/nlp-cour
28
29
30---
31
32# Comparing Tokenizers Side-by-Side 🔬
33
34
```


continued

- Number of tokens varies!

```
21---
22
23# Special Tokens 🎟️
24
25
26    **Most tokenizers add special tokens for specific purposes:**
27
28
29
30
31
32         | [SEP] | Separator between segments | BERT |
33| --- | --- | --- |
34| [PAD] | Padding to same length | Most models |
35| [UNK] | Unknown/rare tokens | Most models |
36| [MASK] | Masked token for training | BERT |
37| <s>, </s> | Start/end of sequence | GPT-2, T5 |
38| <|endoftext|> | Document boundary | GPT-2 |
```

- Number of tokens varies!

```
41
42
43
44    **Example usage:**
45
46        `[CLS] The cat sat [SEP] The dog ran [SEP]`
47
48
49---
50
51# Working with Special Tokens 🔧
52
53
```

...continued

from transformers import AutoTokenizer

Output: "hello world"

```
21
22
23
24    **Sweet spot:** 30k-50k tokens for most models
25
26
27        GPT-2: 50k | BERT: 30k | T5: 32k
28
29
30---
31
32# Tokenization Pitfalls and Gotchas ⚠️
33
34
35    **Common issues to watch out for:**
36
37    1. **Tokenizer-model mismatch**
38        - Always use the tokenizer that matches your model!
```

Output: "hello world"

```
41        - BERT: 512 tokens | GPT-2: 1024 | GPT-3: 2048
42- Text gets truncated if too long!
433. **Case sensitivity**
44        - `bert-base-uncased` lowercases everything
45- `bert-base-cased` preserves case
464. **Rare words → many tokens**
47        - "antidisestablishmentarianism" → 10+ tokens
48- Can hit sequence limit faster than expected!
495. **Special characters**
50        - Emoji, Unicode, accents may be split unexpectedly
51
52
53---
54
55# Debugging Tokenization 🐛
56
57
```